Projetos Práticos de Programação em C

Do Fundamento à Aplicação Real

Um guia prático para estudantes de Sistemas de Informação aplicarem conceitos de structs, ponteiros e alocação dinâmica em projetos de terminal reais.

Versão 1.0 • Foco em Lógica e Estrutura de Dados

1. Introdução

A transição entre o estudo teórico e a prática é o momento mais importante na formação de um desenvolvedor. Este documento foi criado para você, estudante de Sistemas de Informação, que já entende os fundamentos de C e foi introduzido a ferramentas mais avançadas como **structs, typedef, ponteiros, recursão e alocação dinâmica (malloc/realloc/free)**.

O objetivo não é fornecer código pronto, mas sim ideias claras de aplicações de terminal úteis e realistas. Você aprenderá a dividir um problema grande em etapas menores. Caso ainda não tenha estudado manipulação de arquivos (para salvar os dados permanentemente), todas as aplicações podem ser feitas rodando apenas na memória RAM enquanto o programa estiver aberto. Salvar em arquivo é sempre uma evolução natural e opcional.

2. Quadro Comparativo de Projetos

Projeto	Foco Principal (Conceitos)	Dificuldade
1. Gerenciador de Tarefas	Vetores de Structs, Lógica Base	Iniciante
2. Sistema de Estoque	Alocação Dinâmica (Arrays), Realloc	Intermediário
3. Agenda de Contatos	Ponteiros, Listas Encadeadas, Free	Intermediário
4. Fila de Banco	Filas (FIFO), Structs, Ponteiros	Avançado
5. Calculadora de Expressões	Pilhas (LIFO), Recursão	Avançado
6. Chaveamento de Torneio	Árvores, Recursão, Vetores Complexos	Desafio Extra

Projeto 1: Gerenciador de Tarefas (To-Do List)

Problema real: As pessoas precisam organizar suas atividades diárias, saber o que já foi concluído e o que está pendente, sem depender de memória.

Objetivo: Criar um sistema de terminal interativo onde o usuário possa cadastrar, listar e marcar tarefas como concluídas.

Conceitos Praticados

Structs | Typedef | Laços de Repetição | Organização de Menus

Funcionalidades e Evolução

- **Versão Mínima:** Um array estático (ex: limite de 100 tarefas). Uma `struct Tarefa` com ID, descrição e status (0 para pendente, 1 para concluída). Menu simples no terminal.
- **Melhoria Intermediária:** Adicionar a opção de excluir uma tarefa (deslocando os itens do array para não deixar buracos) e editar a descrição.
- **Recursos Extras:** Ordenar as tarefas (ex: mostrar pendentes primeiro). Salvar e ler as tarefas de um arquivo texto (conceito novo opcional).

> **Desafio Principal**
>
> O maior desafio inicial é gerenciar o índice do array estático. Você precisa saber exatamente quantas tarefas existem no momento para não tentar listar lixo da memória e controlar a exclusão corretamente.

Dificuldade Estimada: Iniciante. Ótimo para aquecer e validar o entendimento básico de structs.

Projeto 2: Sistema de Estoque Dinâmico

Problema real: Lojas precisam saber a quantidade de produtos disponíveis, mas o número de produtos diferentes no catálogo pode crescer indefinidamente.

Objetivo: Desenvolver um inventário onde o limite de produtos não é fixo no código, crescendo sob demanda conforme novos itens são cadastrados.

Conceitos Praticados

Structs | Ponteiros | Malloc / Realloc | Free | Gerenciamento de Memória

Funcionalidades e Evolução

- **Versão Mínima:** Usar `malloc` para alocar um array inicial de structs (ex: 5 espaços). Permitir adicionar produto (código, nome, quantidade). Quando atingir 5, exibir um aviso.
- **Melhoria Intermediária:** Implementar `realloc`. Se o usuário tentar adicionar o 6º produto, o sistema dobra o tamanho do array alocado na memória sem perder os dados anteriores.
- **Recursos Extras:** Função para registrar "venda" (diminui quantidade) ou "reposição" (aumenta quantidade). Alertas para produtos com estoque baixo.

> **Desafio Principal**
>
> O uso seguro do `realloc`. É crucial não perder a referência do ponteiro original caso a realocação falhe. Ao final da execução, é fundamental usar o `free` para devolver a memória ao sistema operacional.

Dificuldade Estimada: Intermediário. Introduz responsabilidade real com a memória do computador.

Projeto 3: Agenda de Contatos Inteligente

Problema: Agendas tradicionais em array sofrem com lentidão ao inserir contatos no meio da ordem alfabética (exige deslocar tudo).

Objetivo: Criar um gerenciador de contatos que utilize uma lista encadeada, permitindo inserção e remoção flexível e eficiente.

Conceitos Praticados

Structs auto-referenciadas | Ponteiros | Malloc/Free | Listas Encadeadas

- **Versão Mínima:** Criar uma struct com dados do contato e um ponteiro `proximo`. Implementar a inserção de contatos sempre no final da lista e a listagem completa no terminal.
- **Melhoria Intermediária:** Adicionar função de busca por nome e remoção de um nó específico (requer religar os ponteiros do contato anterior e posterior).
- **Extras:** Inserção já em ordem alfabética no momento do cadastro.

> **Desafio:** Manusear corretamente os ponteiros durante a exclusão para evitar os famosos "Segmentation Faults" (falha de segmentação).

Projeto 4: Simulador de Fila de Banco

Problema: Sistemas de atendimento precisam garantir que a ordem de chegada seja estritamente respeitada, separando prioridades.

Objetivo: Construir um sistema de senhas simulando caixas de atendimento, aplicando a lógica de Filas (FIFO - First In, First Out).

Conceitos Praticados

Ponteiros | Alocação Dinâmica | Filas (FIFO)

- **Versão Mínima:** Uma fila única de pessoas (struct com nome e número da senha). O menu permite "Gerar Senha" (enfileirar) e "Chamar Próximo" (desenfileirar e liberar memória).
- **Melhoria Intermediária:** Criar duas filas: "Normal" e "Prioritária". O sistema deve chamar 2 prioritários para cada 1 normal.

Roteiro de Desenvolvimento

Para não se sentir sobrecarregado, siga este roteiro universal ao construir qualquer um dos projetos acima. O segredo da programação em C não está em escrever tudo de uma vez, mas em validar pequenos passos.

1. **Desenho no Papel:** Antes de abrir o editor, desenhe como sua `struct` vai se parecer e o que o menu do terminal deve mostrar. Entenda o fluxo dos dados.
2. **Esqueleto do Programa:** Crie a função `main`, inclua as bibliotecas (`stdio.h`, `stdlib.h`, `string.h`) e monte um laço `while` simples usando um `switch-case` para o menu. Teste apenas se o menu aparece e fecha corretamente.
3. **Versão Mínima (MVP):** Implemente apenas a funcionalidade de "Adicionar" e "Listar". Imprima mensagens na tela para garantir que as variáveis estão guardando os valores corretos.
4. **Funcionalidades Complexas:** Só após o MVP funcionar, implemente edição e exclusão. Faça testes manuais intensivos (ex: tente excluir o último item da lista e veja se o programa quebra).
5. **Lapidação Visual e Tratamento de Erros:** Limpe a tela entre os menus (usando comandos de sistema, se desejado), valide se o usuário digitou letras onde deveria ser número.
6. **Publicação no GitHub:** Crie um repositório, adicione seu código em `.c` e escreva um arquivo `README.md` simples explicando o que o projeto faz e quais conceitos da faculdade você aplicou. É um excelente portfólio inicial!

Como escolher seu primeiro projeto?

Se você se sente inseguro com a sintaxe: **Gerenciador de Tarefas**.
Se você quer dominar a gestão de memória (malloc/realloc): **Sistema de Estoque**.
Se você quer treinar lógica de ponteiros e estruturas encadeadas: **Agenda ou Fila de Banco**.